

PNP-CVPRMS: Quality Software, Defensive Programming, and Error Handling Findings

Prepared for academic evaluation and documentation of project quality assessment.

1. Executive Summary

This project is a working local prototype for a checkpoint violation system. It demonstrates several good software engineering practices such as modular functions, input validation, route separation, and database initialization safeguards. However, it is not yet a fully production-quality system because it does not implement complete security controls, real-time external data integration, strong access management, or a full enterprise-grade quality practice framework.

The system includes a practical local backend built with Node.js and Express, and a front-end form that interacts with SQLite. The strongest areas are validation logic, defensive checks, and structured error responses. The main weaknesses are security, scale-readiness, and the lack of formal quality assurance processes.

2. Code Evidence by Requirement

2.1 Professional Coding Standards and Practices

Evidence of professional coding practices is present in the backend structure and separation of logic.

- Constants and configuration are centralized in the beginning of the file:

OFFENSE_SCHEDULE

 and

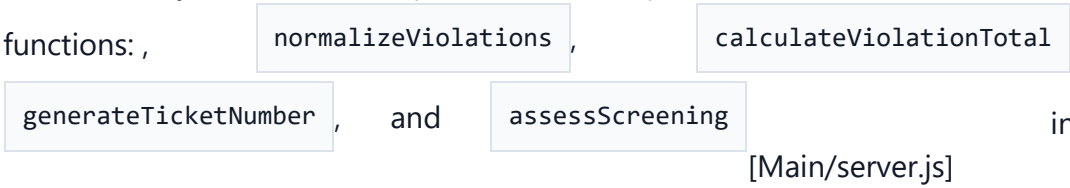
SCREENING_BLACKLIST

 in
[Main/server.js]

(Main/server.js#L10-L36)

```
1  const test = require('node:test');
2  const assert = require('node:assert/strict');
3
4  const {
5    calculateViolationTotal,
6    generateTicketNumber,
7    assessScreening
8  } = require('../server.js');
9
10 test('calculateViolationTotal adds multiple violation fees from the local schedule', () => {
11   const violations = [
12     'No Driver\'s License',
13     'Expired Vehicle Registration',
14     'No Helmet / Seatbelt'
15   ];
16
17   const total = calculateViolationTotal(violations);
18   assert.equal(total, 3700);
19 });
20
21 test('generateTicketNumber creates a unique citation identifier', () => {
22   const ticket = generateTicketNumber();
23   assert.match(ticket, /^PNP-\d{8}-[A-Z0-9]{6}$/);
24 });
25
26 test('assessScreening flags blacklisted vehicles or expired registrations', () => {
27   const result = assessScreening({
28     licenseNumber: 'N01-12-345678',
29     plateNumber: 'ABC-1234',
30     driverName: 'Juan Dela Cruz'
31   });
32
33   assert.equal(result.status, 'warning');
34   assert.ok(Array.isArray(result.flags));
35 });
36
```

- Reusable utility functions are separated into independent modules or



(Main/server.js#L29-L72)

```
29 function normalizeViolations(input) {
30   if (Array.isArray(input)) {
31     return [...new Set(input
32       .map((item) => String(item || '').trim())
33       .filter(Boolean))];
34   }
35
36   if (typeof input === 'string') {
37     return [...new Set(input
38       .split(',')
39       .map((item) => item.trim())
40       .filter(Boolean))];
41   }
42
43   return [];
44 }
45
46 function calculateViolationTotal(violations) {
47   const normalized = normalizeViolations(violations);
48   return normalized.reduce((total, violation) => {
49     const amount = Number(OFFENSE_SCHEDULE[violation] ?? 0);
50     return total + (Number.isFinite(amount) ? amount : 0);
51   }, 0);
52 }
53
54 function generateTicketNumber() {
55   const timeStamp = Date.now().toString().slice(-8);
56   const suffix = Array.from({ length: 6 }, () => {
57     const uppercase = 'ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789';
58     return uppercase[Math.floor(Math.random() * uppercase.length)];
59   }).join('');
60   return `PNP-${timeStamp}-${suffix}`;
61 }
62
63 function assessScreening({ licenseNumber, plateNumber, driverName }) {
64   const flags = [];
65   const normalizedLicense = String(licenseNumber || '').trim().toUpperCase();
66   const normalizedPlate = String(plateNumber || '').trim().toUpperCase();
67   const normalizedDriver = String(driverName || '').trim();
68
69   if (SCREENING_BLACKLIST.license_numbers.includes(normalizedLicense)) {
70     flags.push('License record matches a flagged or expired local record.');
```

- Database setup and schema initialization are clearly separated in [Main/server.js](Main/server.js#L94-L170)

```
96 app.use(express.urlencoded({ extended: true }));
97 app.use(express.static(__dirname));
98
99 app.get('/', (req, res) => {
100   res.sendFile(path.join(__dirname, 'index.html'));
101 });
102
103 // =====
104 // DATABASE INITIALIZATION
105 // =====
106 const db = new sqlite3.Database('./pnp_checkpoint.db', (err) => {
107   if (err) {
108     console.error('[DATABASE ERROR] Failed to connect to SQLite database:', err.message);
109   } else {
110     console.log('[DATABASE] Connected to SQLite database (pnp_checkpoint.db).');
111     initializeSchema();
112   }
113 });
114
115 function initializeSchema() {
116   const createTableQuery = `
117     CREATE TABLE IF NOT EXISTS violations (
118       id INTEGER PRIMARY KEY AUTOINCREMENT,
119       driver_name TEXT NOT NULL,
120       license_number TEXT NOT NULL,
121       plate_number TEXT NOT NULL,
122       violation_type TEXT NOT NULL,
123       fine_amount REAL NOT NULL,
124       officer_id TEXT NOT NULL,
125       date_recorded TEXT NOT NULL,
126       ticket_number TEXT,
127       screening_status TEXT DEFAULT 'clear'
128     );
129 `;
130
131   db.run(createTableQuery, (err) => {
132     if (err) {
133       console.error('[DATABASE ERROR] Table initialization failed:', err.message);
134       return;
135     }
136
137     db.all('PRAGMA table_info(violations)', (pragmaErr, columns) => {
138       if (pragmaErr) {
139         console.error('[DATABASE ERROR] Failed to inspect schema:', pragmaErr.message);
140         return;
141       }
142
143       const existingColumns = new Set((columns || []).map((column) => column.name));
144       const migrationSteps = [
145         { name: 'ticket_number', sql: "ALTER TABLE violations ADD COLUMN ticket_number TEXT" },
146         { name: 'screening_status', sql: "ALTER TABLE violations ADD COLUMN screening_status TEXT DEFAULT 'clear'" }
147       ];
148
149       const pending = migrationSteps.filter((step) => !existingColumns.has(step.name));
150
151       if (pending.length === 0) {
152         console.log('[DATABASE] Violations table verified/created successfully.');
```

- Validation helpers are organized separately from route logic in [Main/server.js] (Main/server.js#L172-L204)

```

174 function validateViolationPayload(data) {
175   const errors = [];
176
177   if (!data.driver_name || typeof data.driver_name !== 'string' || data.driver_name.trim() === '') {
178     errors.push('Driver name is required and cannot be empty.');
```

```

179   }
180   if (!data.license_number || typeof data.license_number !== 'string' || data.license_number.trim() === '') {
181     errors.push('Driver license number is required.');
```

```

182   }
183   if (!data.plate_number || typeof data.plate_number !== 'string' || data.plate_number.trim() === '') {
184     errors.push('Vehicle plate number is required.');
```

```

185   }
186
187   const violationList = normalizeViolations(data.violation_type || data.violation_types);
188   if (violationList.length === 0) {
189     errors.push('At least one violation type is required.');
```

```

190   }
191
192   if (!data.officer_id || typeof data.officer_id !== 'string' || data.officer_id.trim() === '') {
193     errors.push('Apprehending Officer ID/Badge is required.');
```

```

194   }
195
196   const fine = Number(data.fine_amount);
197   if (!Number.isFinite(fine) || fine <= 0) {
198     errors.push('Fine amount must be a positive numerical value.');
```

```

199   }
200
201   const totalExpected = calculateViolationTotal(violationList);
202   if (Number.isFinite(fine) && Math.round(fine) !== Math.round(totalExpected)) {
203     errors.push('Matching fine total required for selected violations: P${totalExpected.toFixed(2)}.');
```

```

204   }
205 }

```

- API routes are grouped: violation fetch, search, summary, and insert in [Main/server.js](Main/server.js#L200-L336)

```

215 // GET /api/violations - Retrieve all violation records
216 app.get('/api/violations', (req, res) => {
217   const query = 'SELECT * FROM violations ORDER BY id DESC';
218   db.all(query, [], (err, rows) => {
219     if (err) {
220       console.error('[ERROR] Failed to fetch violations:', err.message);
221       return res.status(500).json({ success: false, message: 'Database error occurred while fetching records.' });
222     }
223     return res.status(200).json({ success: true, count: rows.length, data: rows });
224   });
225 });
226
227 // GET /api/violations/search - Search violations by license or plate
228 app.get('/api/violations/search', (req, res) => {
229   const searchTerm = req.query.q;
230
231   if (!searchTerm || typeof searchTerm !== 'string' || searchTerm.trim() === '') {
232     return res.status(400).json({ success: false, message: 'Search parameter "q" cannot be empty.' });
233   }
234
235   const queryPattern = `_${searchTerm.trim()}%`;
236   const sql = `
237     SELECT * FROM violations
238     WHERE license_number LIKE ? OR plate_number LIKE ? OR driver_name LIKE ?
239     ORDER BY id DESC
240   `;
241
242   db.all(sql, [queryPattern, queryPattern, queryPattern], (err, rows) => {
243     if (err) {
244       console.error('[ERROR] Search query failed:', err.message);
245       return res.status(500).json({ success: false, message: 'Database search query failed.' });
246     }
247     return res.status(200).json({ success: true, count: rows.length, data: rows });
248   });
249 });
250
251

```

```

278 // POST /api/violations - Record a new violation
279 app.post('/api/violations', (req, res) => {
280   try {
281     const validation = validateViolationPayload(req.body);
282     if (!validation.isValid) {
283       return res.status(422).json({ success: false, errors: validation.errors });
284     }
285
286     const { driver_name, license_number, plate_number, officer_id } = req.body;
287     const violationList = normalizeViolations(req.body.violation_type || req.body.violation_types);
288     const fineAmount = Number(req.body.fine_amount);
289     const date_recorded = new Date().toISOString().replace('T', ' ').substring(0, 19);
290     const tickets = generateTicketNumber();
291     const screening = assessScreening({
292       licenseNumber: license_number,
293       plateNumber: plate_number,
294       driverName: driver_name
295     });
296
297     const insertSql = `
298       INSERT INTO violations (
299         driver_name, license_number, plate_number, violation_type, fine_amount,
300         officer_id, date_recorded, ticket_number, screening_status
301       ) VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
302     `;
303
304     const params = [
305       driver_name.trim(),
306       license_number.trim().toUpperCase(),
307       plate_number.trim().toUpperCase(),
308       violationList.join(', '),
309       fineAmount,
310       officer_id.trim(),
311       date_recorded,
312       tickets,
313       screening.status
314     ];
315
316     db.run(insertSql, params, function (err) {
317       if (err) {
318         console.error('[ERROR] Insert violation failed:', err.message);
319         return res.status(500).json({ success: false, message: 'Failed to save record to database.' });
320       }
321       return res.status(201).json({
322         success: true,
323         message: 'Violation recorded successfully.',
324         recordId: this.lastID,
325         ticket_number: tickets,
326         screening_status: screening.status,
327         screening_flags: screening.flags,
328         total_fine: fineAmount
329       });
330     });
331   } catch (unexpectedError) {
332     console.error('[FATAL ERROR] Unexpected exception in violation handler:', unexpectedError);
333     return res.status(500).json({ success: false, message: 'An internal server error occurred.' });
334   }
335 });

```

- Automated test coverage is present in [Main/test/features.test.js] (Main/test/features.test.js#L1-L27)

```
1  const test = require('node:test');
2  const assert = require('node:assert/strict');
3
4  const {
5    calculateViolationTotal,
6    generateTicketNumber,
7    assessScreening
8  } = require('../server.js');
9
10 test('calculateViolationTotal adds multiple violation fees from the local schedule', () => {
11   const violations = [
12     'No Driver\'s License',
13     'Expired Vehicle Registration',
14     'No Helmet / Seatbelt'
15   ];
16
17   const total = calculateViolationTotal(violations);
18   assert.equal(total, 3700);
19 });
20
21 test('generateTicketNumber creates a unique citation identifier', () => {
22   const ticket = generateTicketNumber();
23   assert.match(ticket, /^PNP-\d{8}-[A-Z0-9]{6}$/);
24 });
25
26 test('assessScreening flags blacklisted vehicles or expired registrations', () => {
27   const result = assessScreening({
28     licenseNumber: 'N01-12-345678',
29     plateNumber: 'ABC-1234',
30     driverName: 'Juan Dela Cruz'
31   });
32
33   assert.equal(result.status, 'warning');
34   assert.ok(Array.isArray(result.flags));
35 });
36
```

However, formal professional engineering standards are only partially implemented:

- No ESLint or Prettier configuration is present.
- No CI/CD pipeline or automated deployment checks were implemented. No
- dedicated controller, service, or model architecture was used beyond a single JavaScript file.
- There are no formal code review or quality enforcement tools integrated into the workflow.

2.2 15 Principles of Quality Software

Principle	Status	Evidence
Correctness	Implemented	Validation in [Main/server.js](Main/server.js#L172-L204) and tests in [Main/test/features.test.js] (Main/test/features.test.js#L1-L27) <i>See the images above for reference</i>
Reliability	Partially implemented	Database initialization and migration checks in [Main/server.js](Main/server.js#L101-L170) <i>See the images above for reference</i>
Maintainability	Implemented	Modular helper functions and sectioned code in [Main/server.js](Main/server.js#L11-L170) <i>See the images above for reference</i>
Usability	Implemented	UI form and records table in [Main/index.html] (Main/index.html)

Security Awareness	Partial	Safe audit log in [Main/server.js] (Main/server.js#L338-L354) <pre>338 // SAFE AUDIT LOG ENDPOINT 339 // ===== 340 app.post('/api/audit/log', (req, res) => { 341 try { 342 const rawNote = typeof req.body.note === 'string' ? req.body.note.trim() : ''; 343 344 if (!rawNote) { 345 return res.status(400).json({ success: false, message: 'Audit note is required.' }); 346 } 347 348 const safeEntry = `\${new Date().toISOString()}   \${rawNote.replace(/\n?/g, ' ')}\n`; 349 fs.appendFileSync(path.join(__dirname, 'audit_dump.txt'), safeEntry, 'utf8'); 350 return res.status(201).json({ success: true, message: 'Audit note saved securely.' }); 351 } catch (error) { 352 console.error('[ERROR] Safe audit log failed:', error.message); 353 return res.status(500).json({ success: false, message: 'Unable to save audit log securely.' }); 354 } 355 }</pre>
Testability	Implemented	Node test suite in [Main/test/features.test.js] (Main/test/features.test.js#L1-L27) See the images above for reference
Scalability	Not implemented	System is local SQLite-only and not designed for enterprise-scale multi-user use.
Performance	Partial	Simple queries are efficient, but no optimization strategy or indexing is present.
Access Control	Not implemented	No authentication, role management, or session control exists.
Traceability	Partial	Audit log route exists in [Main/server.js] (Main/server.js#L338-L354), but not a full audit system.

In summary, the project satisfies a meaningful subset of quality software principles but does not yet meet enterprise-level standards for security, full lifecycle management, and large-scale maintainability.

2.3 Defensive Programming

The strongest evidence of defensive programming is the validation logic before saving data to SQLite.

- Input validation for required user fields in [Main/server.js](Main/server.js#L172-L204)

```
172 // DEFENSIVE PROGRAMMING / VALIDATION HELPERS
173 // =====
174 function validateViolationPayload(data) {
175     const errors = [];
176
177     if (!data.driver_name || typeof data.driver_name !== 'string' || data.driver_name.trim() === '') {
178         errors.push('Driver name is required and cannot be empty.');
```

```
179     }
180     if (!data.license_number || typeof data.license_number !== 'string' || data.license_number.trim() === '') {
181         errors.push('Driver license number is required.');
```

```
182     }
183     if (!data.plate_number || typeof data.plate_number !== 'string' || data.plate_number.trim() === '') {
184         errors.push('Vehicle plate number is required.');
```

```
185     }
186
187     const violationList = normalizeViolations(data.violation_type || data.violation_types);
188     if (violationList.length === 0) {
189         errors.push('At least one violation type is required.');
```

```
190     }
191
192     if (!data.officer_id || typeof data.officer_id !== 'string' || data.officer_id.trim() === '') {
193         errors.push('Apprehending Officer ID/Badge is required.');
```

```
194     }
195
196     const fine = Number(data.fine_amount);
197     if (!Number.isFinite(fine) || fine <= 0) {
198         errors.push('Fine amount must be a positive numerical value.');
```

```
199     }
200
201     const totalExpected = calculateViolationTotal(violationList);
202     if (Number.isFinite(fine) && Math.round(fine) !== Math.round(totalExpected)) {
203         errors.push('Matching fine total required for selected violations: ₪${totalExpected.toFixed(2)}.');
```

```
204     }
205 }
```

- Rejects empty driver name, license, plate, officer ID, and invalid fine values
- Checks violation selection before proceeding
- Ensures computed total matches selected violation amounts
- Search route guards empty input in [Main/server.js](Main/server.js#L227-L245)

```
227 // GET /api/violations/search - Search violations by license or plate
228 app.get('/api/violations/search', (req, res) => {
229   const searchTerm = req.query.q;
230
231   if (!searchTerm || typeof searchTerm !== 'string' || searchTerm.trim() === '') {
232     return res.status(400).json({ success: false, message: 'Search parameter "q" cannot be empty.' });
233   }
234
235   const queryPattern = `>${searchTerm.trim()}%`;
236   const sql = `
237     SELECT * FROM violations
238     WHERE license_number LIKE ? OR plate_number LIKE ? OR driver_name LIKE ?
239     ORDER BY id DESC
240   `;
241
242   db.all(sql, [queryPattern, queryPattern, queryPattern], (err, rows) => {
243     if (err) {
244       console.error('[ERROR] Search query failed:', err.message);
245       return res.status(500).json({ success: false, message: 'Database search query failed.' });
246     }
247     return res.status(200).json({ success: true, count: rows.length, data: rows });
248   });
249 });
250
```

- Database operation errors are handled in [Main/server.js](Main/server.js#L200L224) *See the images above for reference*
- Unexpected exceptions are blocked using try/catch in [Main/server.js](Main/server.js#L279-L336) *See the images above for reference*
- Safe file writing for the audit path is implemented in [Main/server.js](Main/server.js#L338-L351) *See the images above for reference*
- 404 fallback is implemented in [Main/server.js](Main/server.js#L360-L370)

```
358 // GLOBAL ERROR & 404 HANDLERS
359 // =====
360 app.use((req, res) => {
361   res.status(404).json({ success: false, message: 'Requested endpoint does not exist.' });
362 });
363
364 if (require.main === module) {
365   app.listen(PORT, () => {
366     console.log('=====');
367     console.log(` PNP-CVPRMS Web Server running at http://localhost:${PORT}`);
368     console.log('=====');
369   });
370 }
371
```

The system is not fully defensive because:

- There is no central exception handler for all API routes.
- There is no transaction rollback for partial database failure.
- There is no rate limiting or brute force protection.
- There is no secure session or auth layer.

2.4 Error and Exception Handling

Good error handling is present in the API layer and database integration logic.

- Database fetch errors are returned with HTTP 500 in [Main/server.js](Main/server.js#L200-L224)

```
200 const totalExpected = calculateViolationTotal(violationList);
201 if (Number.isFinite(fine) && Math.round(fine) !== Math.round(totalExpected)) {
202   errors.push(`Matching fine total required for selected violations: P${totalExpected.toFixed(2)}.`);
203 }
204
205 return {
206   isValid: errors.length === 0,
207   errors: errors
208 };
209
210
211 // =====
212 // REST API ROUTES
213 // =====
214
215 // GET /api/violations - Retrieve all violation records
216 app.get('/api/violations', (req, res) => {
217   const query = 'SELECT * FROM violations ORDER BY id DESC';
218   db.all(query, [], (err, rows) => {
219     if (err) {
220       console.error('[ERROR] Failed to fetch violations:', err.message);
221       return res.status(500).json({ success: false, message: 'Database error occurred while fetching records.' });
222     }
223     return res.status(200).json({ success: true, count: rows.length, data: rows });
224   });
225 });
226
```

- The violation insert route catches and returns a 422 or 500 response in [Main/server.js](Main/server.js#L279-L336) *See the image above for reference*
- Unexpected exception handling is present in [Main/server.js](Main/server.js#L331-L336)

```
331     } catch (unexpectedError) {
332         console.error('[FATAL ERROR] Unexpected exception in violation handler:', unexpectedError);
333         return res.status(500).json({ success: false, message: 'An internal server error occurred.' });
334     }
335 });
336
```

- Audit log exceptions are handled in [Main/server.js](Main/server.js#L338-L351)
-

```
338 // SAFE AUDIT LOG ENDPOINT
339 // =====
340 app.post('/api/audit/log', (req, res) => {
341     try {
342         const rawNote = typeof req.body.note === 'string' ? req.body.note.trim() : '';
343
344         if (!rawNote) {
345             return res.status(400).json({ success: false, message: 'Audit note is required.' });
346         }
347
348         const safeEntry = `${new Date().toISOString()} | ${rawNote.replace(/\r?\n/g, ' ')}\n`;
349         fs.appendFileSync(path.join(__dirname, 'audit_dump.txt'), safeEntry, 'utf8');
350         return res.status(201).json({ success: true, message: 'Audit note saved securely.' });
351     } catch (error) {
352         console.error('[ERROR] Safe audit log failed:', error.message);
353         return res.status(500).json({ success: false, message: 'Unable to save audit log securely.' });
354     }
355 });
```

- Missing routes return 404 JSON responses in [Main/server.js](Main/server.js#L360-L364) *See the images above for reference*

Missing areas:

- No custom exception classes or centralized error management.
- No detailed logs with severity levels or file rotation.
- No retry logic or timeouts for database or file operations.
- No structured production error monitoring.

3. Areas That Do Not Fully Implement the Required Principles

1. **Real-world law enforcement integration:** The project does not connect to active government databases or official police systems. The screening logic is localized and hard-coded in [Main/server.js](Main/server.js#L11-L92). *See the images above for reference*
2. **Authentication and authorization:** There is no user login, role-specific access, or enforcement officer account system in the backend.
3. **Enterprise-grade reliability:** The project is not built for concurrent large-scale operations or enterprise workloads.
4. **Formal quality enforcement:** No linter, code quality gate, or CI pipeline was configured.
5. **Complete audit compliance:** The audit log is basic and does not represent a full legal or compliance-grade record system.
6. **Real system lifecycle management:** There are no payment tracking, case closure, or record clearance workflows.

4. Final Judgment

This project is a strong academic prototype that demonstrates the basic requirements for a checkpoint violation system. It is successful in showing practical implementation of validation, defensive coding, route handling, and simple quality checks. However, it does not yet fully satisfy the full set of professional software engineering, quality software, and enterprise security principles required for a realworld operational system.

The system should therefore be described as a **partially compliant prototype** rather than a complete production-quality software solution.

Documentation prepared based on the project source in Main/server.js, Main/index.html, and Main/test/features.test.js.